Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №7

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «Автентифікація та авторизація в ASP.NET Core (JWT)»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Навчитися реалізовувати автентифікацію користувачів на основі JWT (JSON Web Token) в ASP.NET Core Web API, розмежовувати доступ до ендпоінтів за ролями та політиками авторизації, безпечно хешувати паролі.

2. Завдання варіанта №1 («Бібліотека»)

• **Модель користувача:** User (Id, Username, Email, PasswordHash, Role, CreatedAt) у базі даних SQLite.

• **Хешування паролів:** Сервіс PasswordHasher з використанням криптографічного алгоритму PBKDF2 (100 000 ітерацій) з випадковою сіллю (Salt).

• **Ендпоінти автентифікації:**

• POST /api/auth/register - реєстрація нового облікового запису;

• POST /api/auth/login - перевірка облікових даних та видача JWT-токена;

• GET /api/auth/me - перевірка клеймів автентифікованого користувача.

• **Генерація та валідація JWT:**

• Створення токена через JwtSecurityTokenHandler із симетричним підписом HMAC-SHA256;

• Клейми токена: sub (Id), unique_name (Username), email, role (Role), jti (GUID);

• Підключення middleware AddJwtBearer із валідацією емітента (Issuer), аудиторії (Audience), ключа підпису та терміну дії (Lifetime).

• **Рольова авторизація:**

• Роль **`Admin`**: повний доступ до каталогу (перегляд, додавання, видалення книг);

• Роль **`User`**: доступ лише для читання каталогу книг;

• Анонімні користувачі: доступ лише до ендпоінтів реєстрації та входу.

• **Сценарії доступу:**

• Перевірено та зафіксовано поведінку системи для статусів 200 OK, 401 Unauthorized (запит без токена), 403 Forbidden (звичайний User намагається видалити книгу).

3. Таблиця прав доступу до ендпоінтів

HTTP Метод	**Маршрут**	**Опис**	**Дозволена роль / Доступ**
:---:	:---	:---	:---:
`POST`	`/api/auth/register`	Реєстрація нового акаунта	`[AllowAnonymous]` (Усі)
`POST`	`/api/auth/login`	Вхід та отримання токена	`[AllowAnonymous]` (Усі)
`GET`	`/api/auth/me`	Отримання клеймів поточного користувача	`[Authorize]` (Будь-який автентифікований)
`GET`	`/api/books`	Перегляд списку книг	`[Authorize(Roles = "User,Admin")]`
`GET`	`/api/books/{id}`	Перегляд книги за Id	`[Authorize(Roles = "User,Admin")]`
`POST`	`/api/books`	Створення нової книги	`[Authorize(Roles = "Admin")]` (Лише Admin)
`DELETE`	`/api/books/{id}`	Вилучення книги	`[Authorize(Roles = "Admin")]` (Лише Admin)

4. Зразки роботи та статусів автентифікації

4.1. Успішний вхід та отримання токена (`POST /api/auth/login`)


```
{
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidW5pcXVlX25hbWUiOiJhZG1pbiIsI
nJvbGUiOiJBZG1pbiIsImV4cCI6MTc2MDAwMDAwMH0...",
  "expiration": "2026-10-07T18:00:00Z",
  "username": "admin",
```

```
  "role": "Admin"
}
```

4.2. Сценарій `HTTP 401 Unauthorized` (Спроба виклику `GET /api/books` без заголовка Authorization)

Запит:

```
GET /api/books HTTP/1.1
Host: localhost:5000
```

Відповідь:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
```

4.3. Сценарій `HTTP 403 Forbidden` (Користувач `student` з роллю `User` намагається видалити книгу)

Запит:

```
DELETE /api/books/1 HTTP/1.1
Host: localhost:5000
Authorization: Bearer <Токен_Користувача_student_з_роллю_User>
```

Відповідь:

```
HTTP/1.1 403 Forbidden
```

(Пояснення: Токен валідний, користувач автентифікований, але роль User не має дозволу на вилучення ресурсів, доступних лише для Admin).

4.4. Сценарій `HTTP 200 OK` (Той самий запит під токеном користувача `admin` з роллю `Admin`)

```
DELETE /api/books/1 HTTP/1.1
Host: localhost:5000
Authorization: Bearer <Токен_Адміністратора_admin_з_роллю_Admin>
```

Відповідь:

```
HTTP/1.1 204 No Content
```

5. Посилання на репозиторій

• **URL репозиторію:** https://github.com/2005igor2023-crypto/dotnetvntu

• **Папка проєкту:** /07-auth-jwt

6. Відповіді на контрольні питання

1. У чому принципова різниця між автентифікацією та авторизацією?

Відповідь:

• **Автентифікація (Authentication - Хто ти?):** процес перевірки справжності особи користувача на основі наданих облікових даних (логін, пароль, токен).

• **Авторизація (Authorization - Що тобі дозволено?):** процес визначення прав автентифікованого користувача на виконання конкретної дії або доступ до конкретного ресурсу на основі його ролей чи політик.

2. З яких трьох частин складається JWT і що міститься у кожній із них?

Відповідь: JWT складається з трьох частин, розділених крапкою: Header.Payload.Signature:

1. **Header (Заголовок):** JSON у форматі Base64Url, що містить тип токена (JWT) та алгоритм хешування підпису (наприклад, HS256).

2. **Payload (Корисне навантаження):** Base64Url-дані з набором клеймів (ідентифікатор користувача sub, ім'я, роль role, термін придатності exp).

3. **Signature (Цифровий підпис):** криптографічний хеш від з'єднаних Header + Payload, створений із використанням секретного ключа сервера (SecretKey). Підпис гарантує, що дані токена не були підроблені клієнтом.

3. Чому паролі користувачів необхідно зберігати у вигляді хешу, а не у відкритому тексті? Який алгоритм для цього доцільно використати?

Відповідь: Зберігання у відкритому вигляді загрожує масовим витоком облікових записів у разі компрометації бази даних або бекапів. Хеш є незворотною функцією (з нього неможливо відновити вихідний пароль). Для паролів категорично не можна використовувати швидкі алгоритми на кшталт MD5 чи SHA-1/SHA-256 напряму, оскільки вони вразливі до райдужних таблиць (rainbow tables) та брутфорсу на GPU. Необхідно

застосовувати спеціалізовані адаптивні алгоритми повільного хешування із сіллю (Salt): **PBKDF2** (з $\ge 100\,000$ ітерацій), **BCrypt** або **Argon2id**.

4. У чому різниця між кодами статусу 401 Unauthorized та 403 Forbidden?

Відповідь:

• 401 Unauthorized означає, що користувач взагалі **не автентифікований** (токен відсутній, недійсний або його термін дії закінчився).

• 403 Forbidden означає, що користувач **успішно автентифікований**, сервер знає його особу, але його рівень прав (роль/політика) **недостатній** для доступу до запитуваної дії.

5. Чим політикова авторизація (AuthorizationPolicy) гнучкіша за просту перевірку ролі через атрибут?

Відповідь: Проста перевірка ролей ([Authorize(Roles = "Admin")]) є дискретною та статичною. Політики авторизації дозволяють комбінувати довільні вимоги: наявність кількох клеймів одночасно, перевірку віку, перевірку підрозділу, виклик коду перевірки володіння ресурсом (Resource-based authorization - наприклад, «редагувати книгу може лише автор або супер-адмін»), а також динамічне обчислення дозволів під час виконання.

7. Висновки

У ході виконання практичної роботи №7 було реалізовано повноцінну підсистему автентифікації та авторизації на основі JWT для Web API бібліотеки. Застосовано надійне хешування паролів за стандартом PBKDF2, реалізовано сервіс видачі та підпису токенів, налаштовано middleware JwtBearer та розмежовано доступ до операцій за ролями Admin та User. Усі критичні сценарії (200, 401, 403) успішно перевірено та зафіксовано.